CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement: Building a RESTful API with Node.js and Express

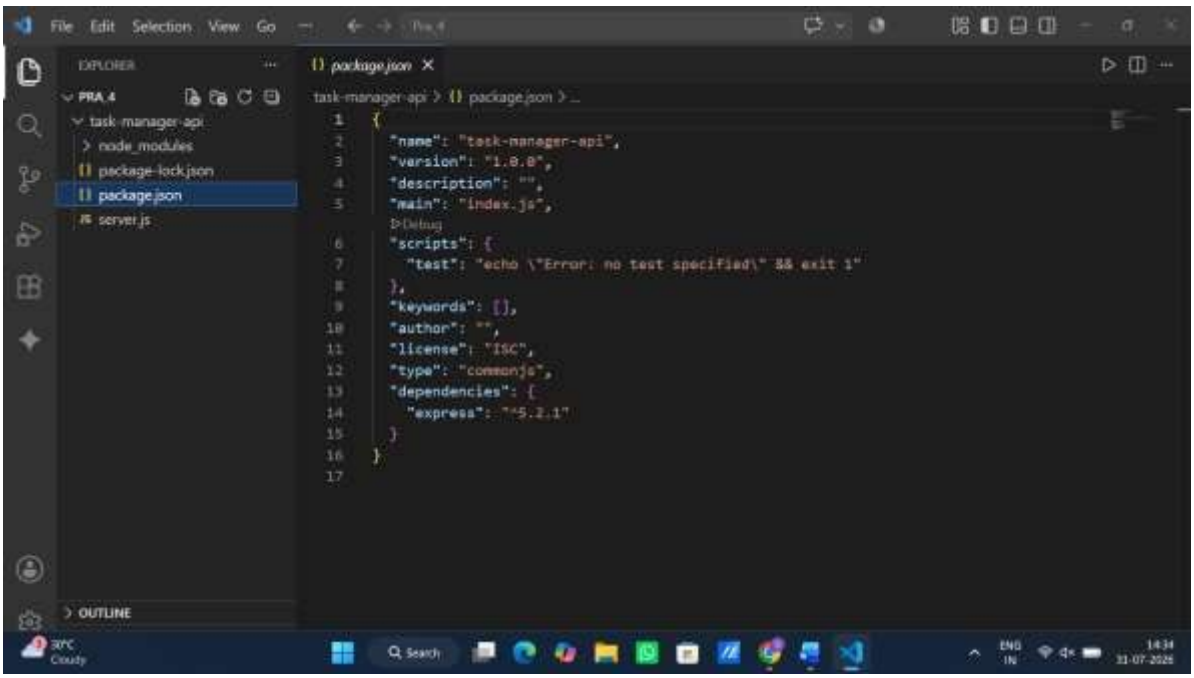
Objective

Build a RESTful Task Management API using **Node.js** and **Express** with:

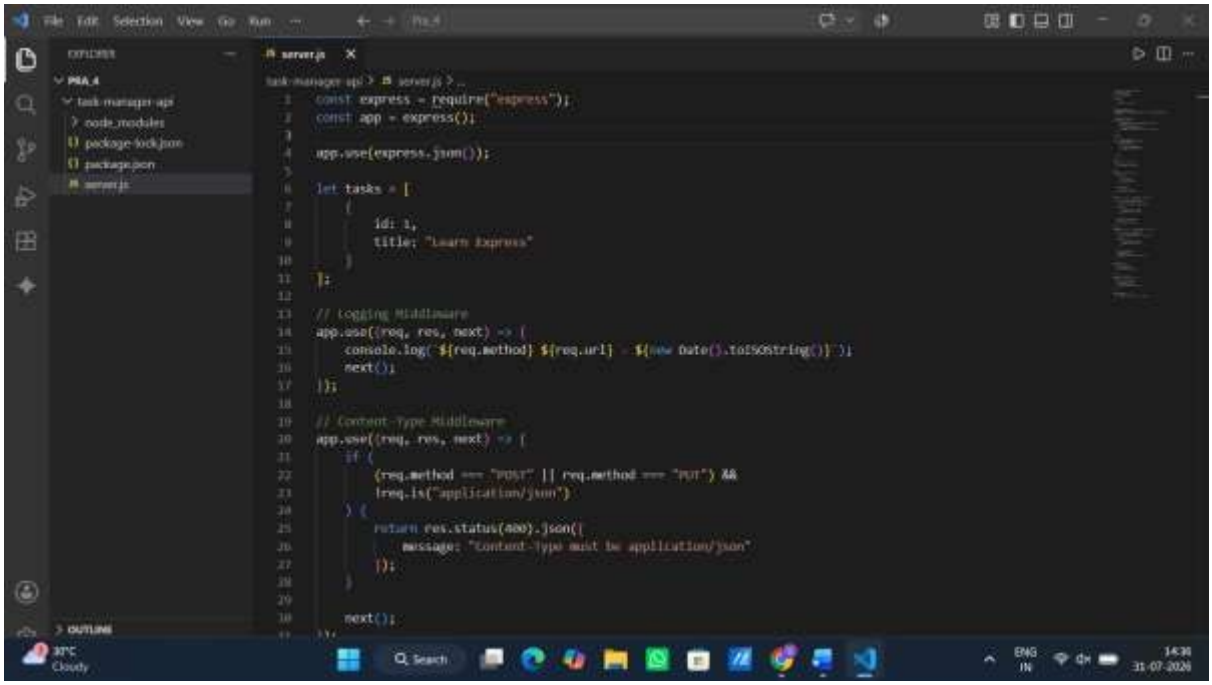
- CRUD Operations
- Logging Middleware
- Global Error Handling
- Proper HTTP Status Codes
- In-Memory Storage (No Database)

Screenshots:

package.json



server.js



```
const express = require("express");
const app = express();

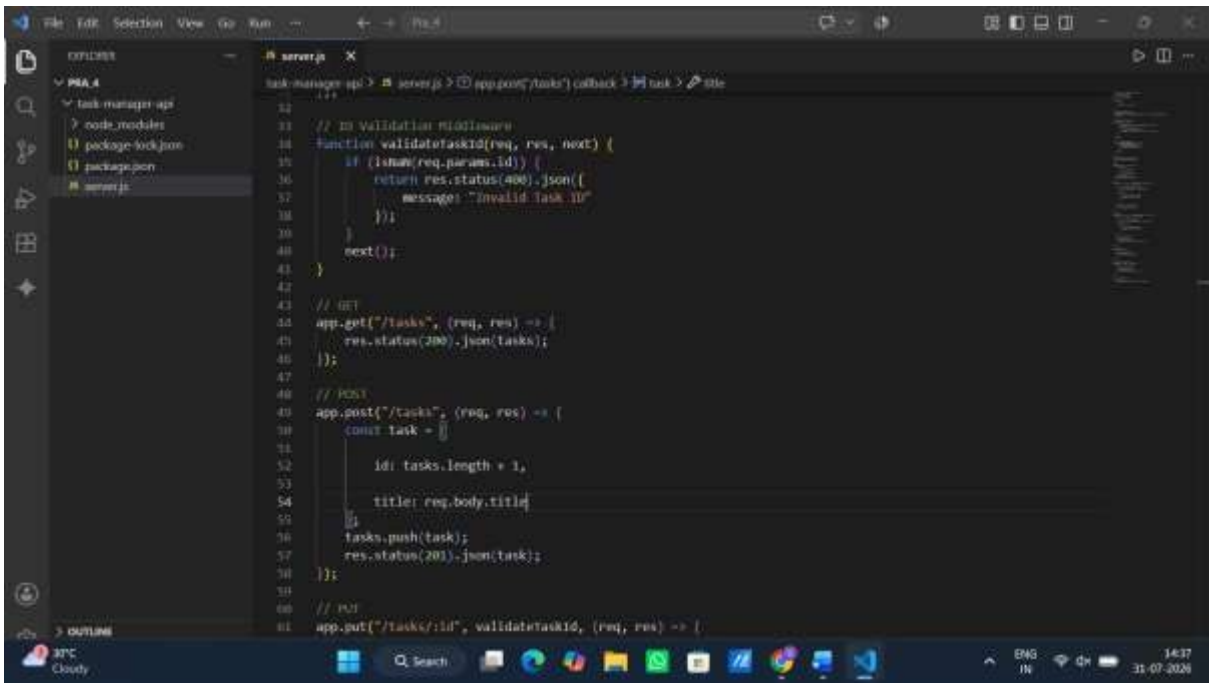
app.use(express.json());

let tasks = [
    {
        id: 1,
        title: "Learn Express"
    }
];

// Logging Middleware
app.use((req, res, next) => {
    console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
    next();
});

// Content-Type Middleware
app.use((req, res, next) => {
    if (
        (req.method === "POST" || req.method === "PUT") &&
        !req.is("application/json")
    ) {
        return res.status(400).json({
            message: "Content-Type must be application/json"
        });
    }

    next();
});
```



```
// ID Validation Middleware
function validateTaskId(req, res, next) {
    if (isNaN(req.params.id)) {
        return res.status(400).json({
            message: "Invalid Task ID"
        });
    }
    next();
}

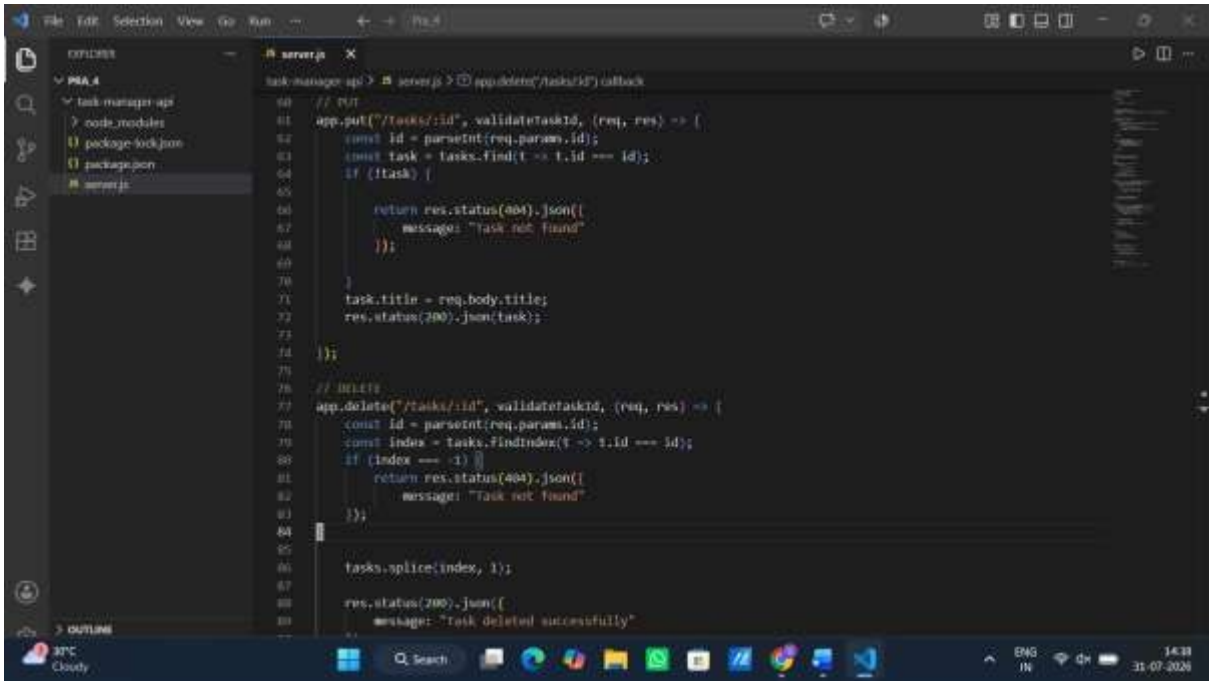
// GET
app.get("/tasks", (req, res) => {
    res.status(200).json(tasks);
});

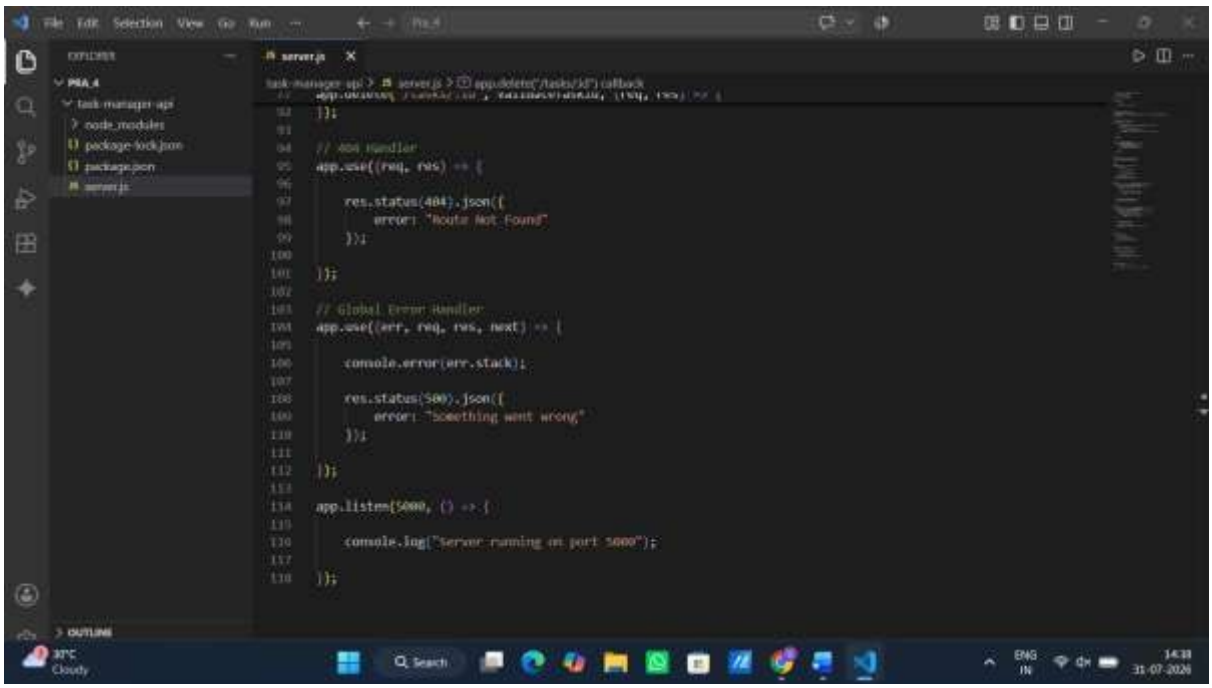
// POST
app.post("/tasks", (req, res) => {
    const task = {

        id: tasks.length + 1,

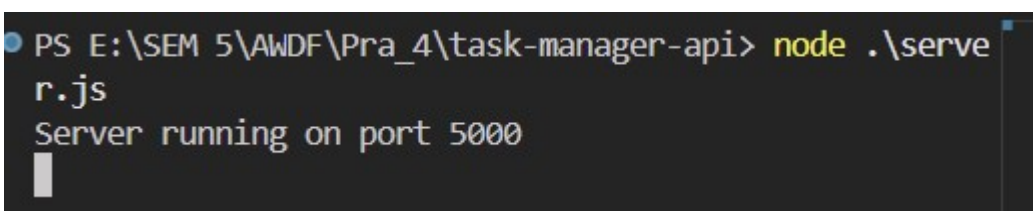
        title: req.body.title
    };
    tasks.push(task);
    res.status(201).json(task);
});

// PUT
app.put("/tasks/:id", validateTaskId, (req, res) => {
```

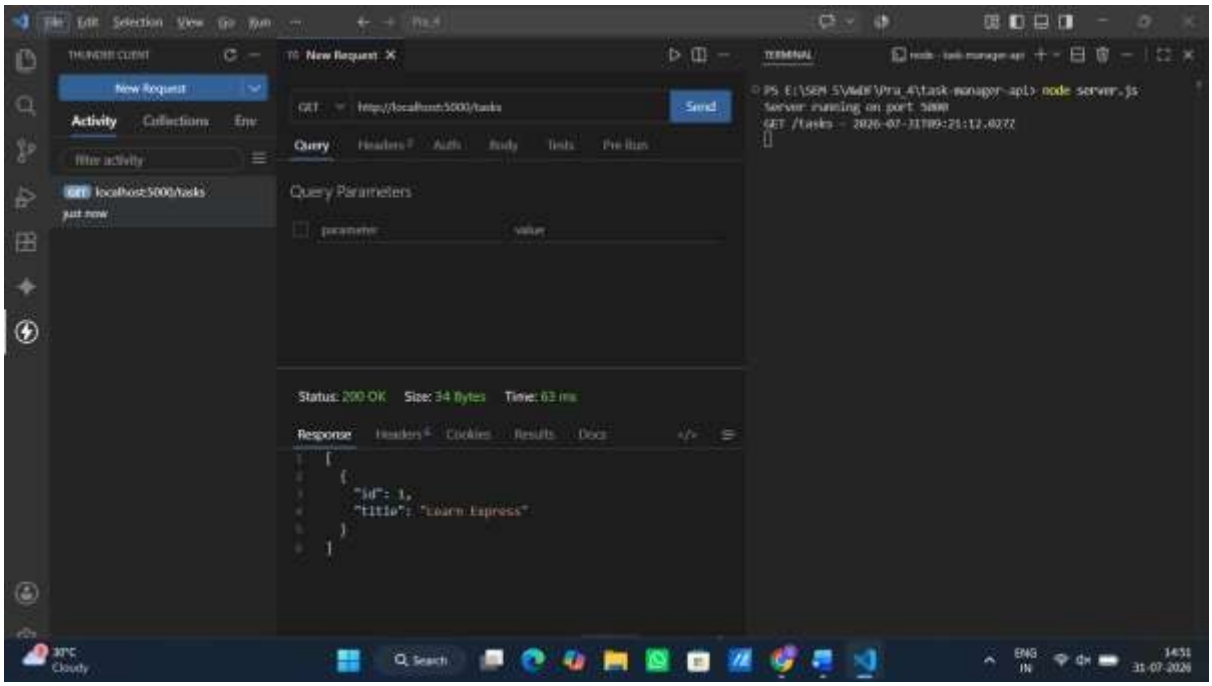

// PUT
app.put("/tasks/:id", validateTaskId, (req, res) => {
const id = parseInt(req.params.id);
const task = tasks.find(t => t.id === id);
if (!task) {
return res.status(404).json({
message: "Task not found"
});
}
task.title = req.body.title;
res.status(200).json(task);
});
// DELETE
app.delete("/tasks/:id", validateTaskId, (req, res) => {
const id = parseInt(req.params.id);
const index = tasks.findIndex(t => t.id === id);
if (index === -1) {
return res.status(404).json({
message: "Task not found"
});
}
tasks.splice(index, 1);
res.status(200).json({
message: "Task deleted successfully"


// 404 Handler
app.use((req, res) => {
res.status(404).json({
error: "Route Not Found"
});
});
// Global Error Handler
app.use((err, req, res, next) => {
console.error(err.stack);
res.status(500).json({
error: "Something went wrong"
});
});
app.listen(5000, () => {
console.log("Server running on port 5000");
});

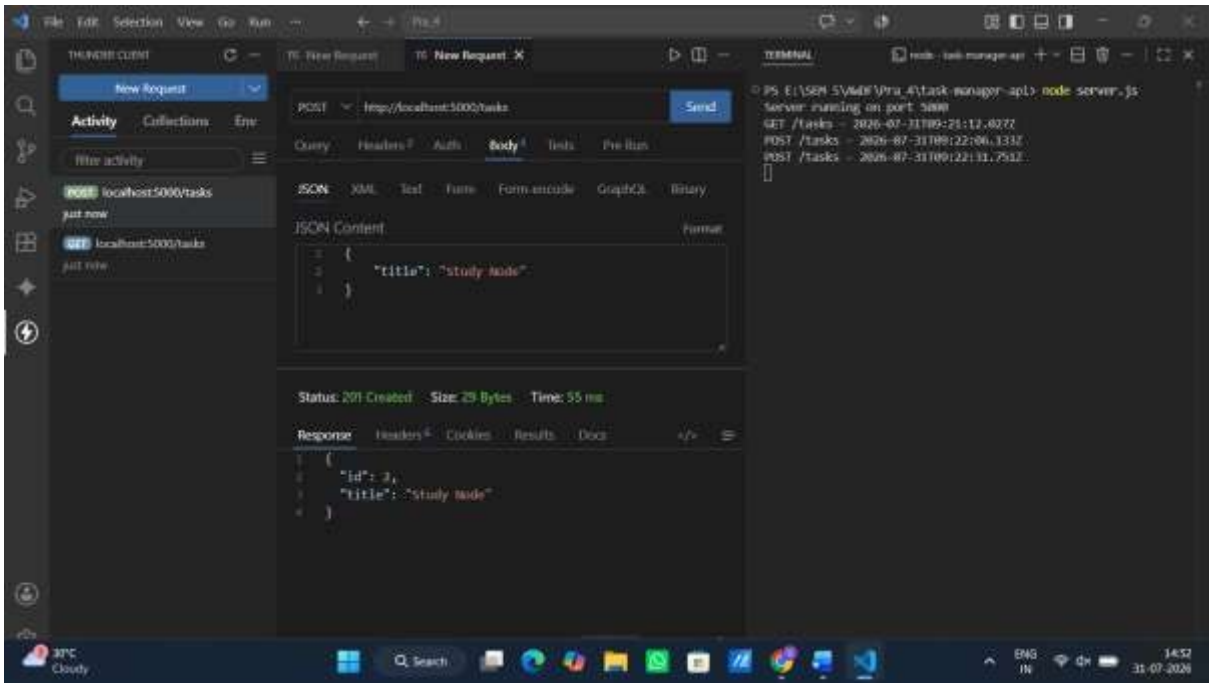
OUTPUT:

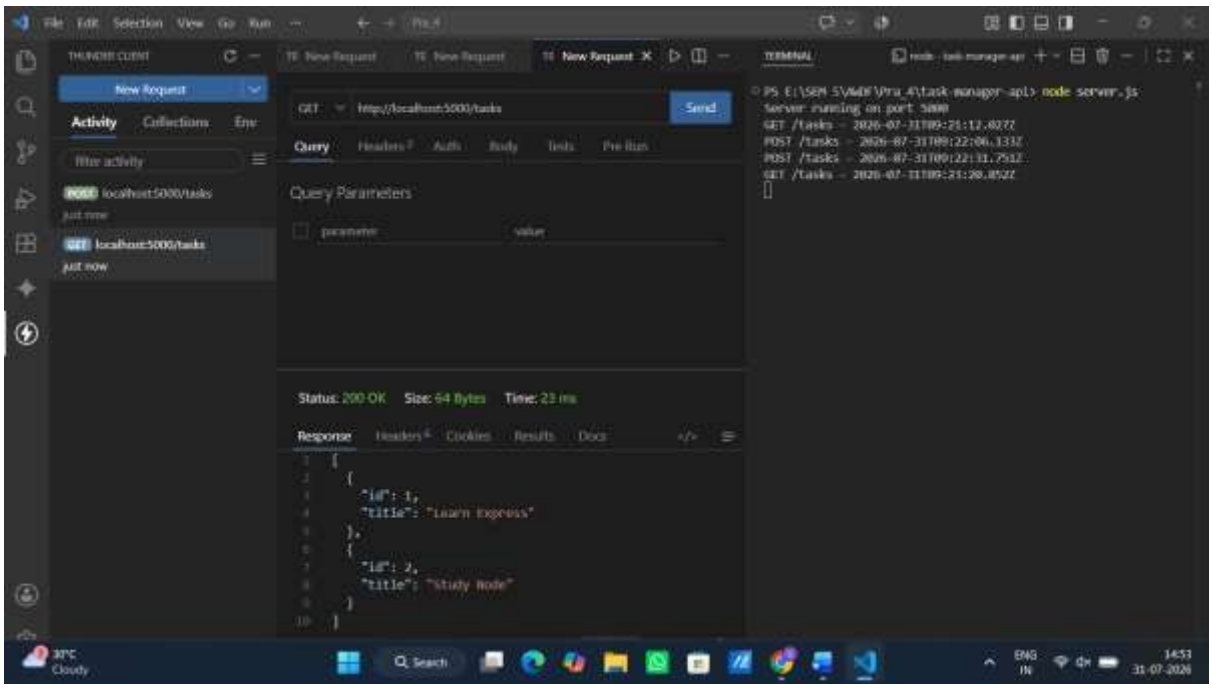

PS E:\SEM 5\AWDF\Pra_4\task-manager-api> node .\server.js
Server running on port 5000

1) GET

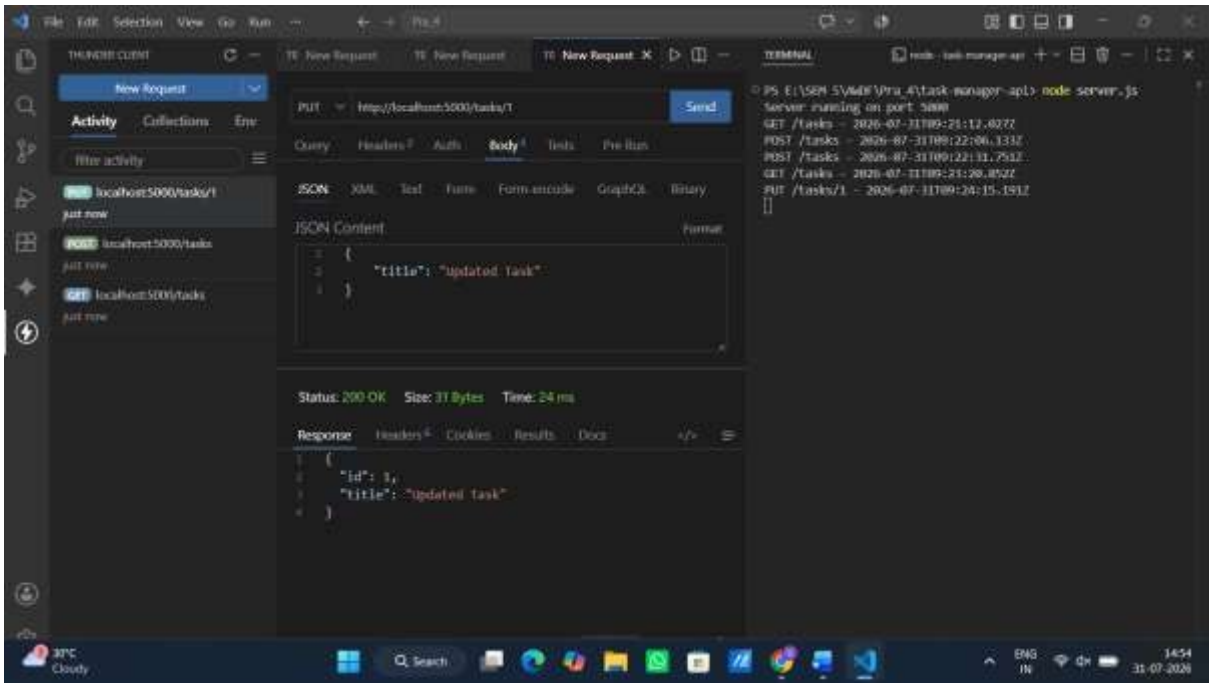

New Request
GET http://localhost:5000/tasks
Send
Query Parameters
Status: 200 OK Size: 34 Bytes Time: 63 ms
Response
{
"id": 1,
"title": "Learn Express"
}
]
PS E:\SEM 5\WebF\Pro_4\task-manager-api> node server.js
Server running on port 5000
GET /tasks - 2026-07-31T09:21:12.027Z

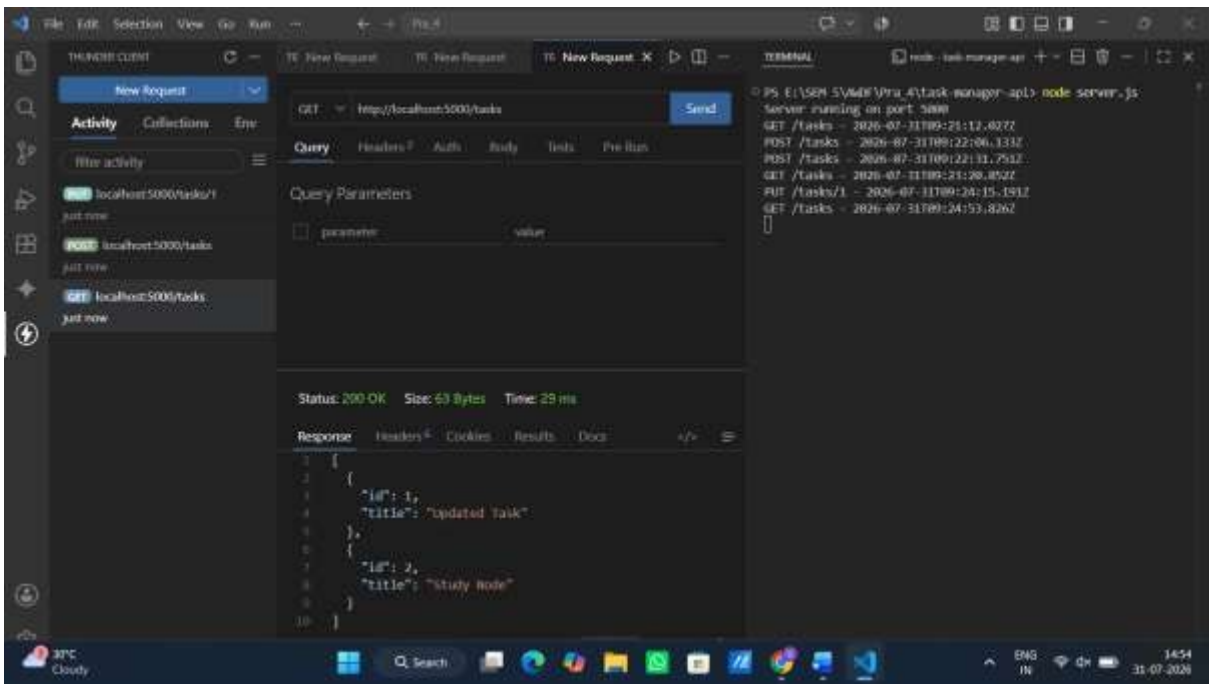
2) **POST**


POST http://localhost:5000/tasks
Send
Body
JSON
{
"title": "Study Node"
}
Status: 201 Created Size: 29 Bytes Time: 55 ms
Response
{
"id": 2,
"title": "Study Node"
}
PS E:\SEM 5\W&DF\Pra_4\task-manager-api> node server.js
Server running on port 5000
GET /tasks - 2026-07-31T09:21:12.027Z
POST /tasks - 2026-07-31T09:22:06.133Z
POST /tasks - 2026-07-31T09:22:31.751Z

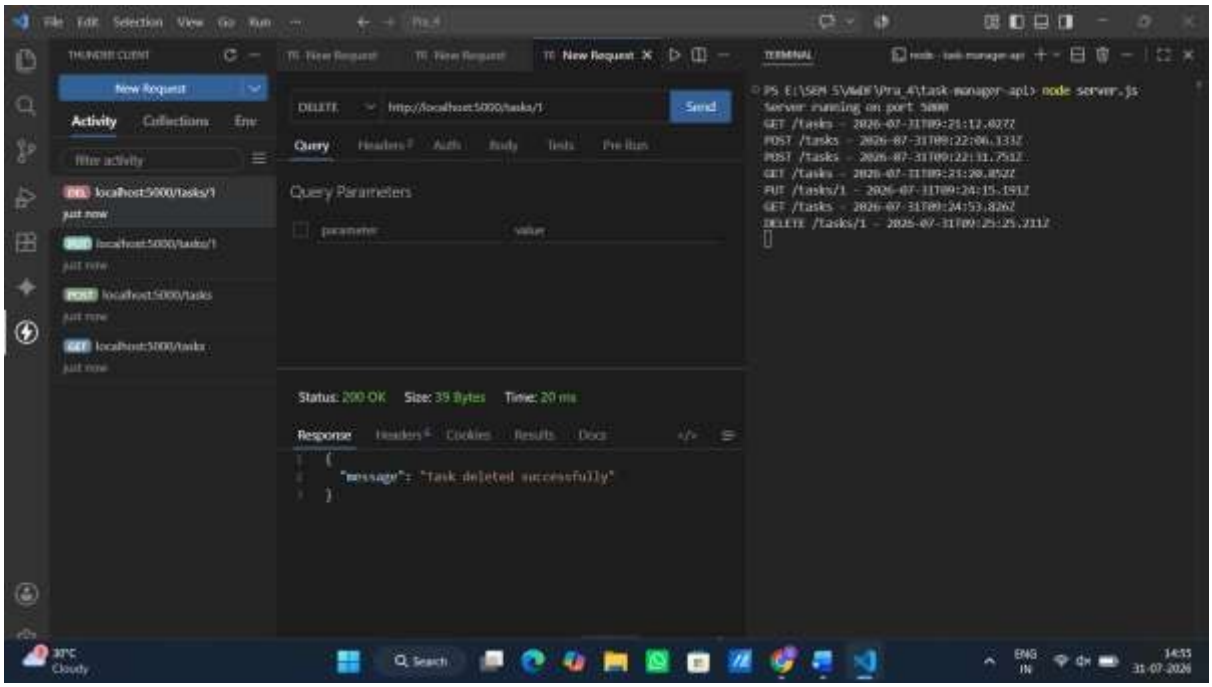

GET http://localhost:5000/tasks
Send
Query
Query Parameters
Status: 200 OK Size: 64 Bytes Time: 23 ms
Response
[
{
"id": 1,
"title": "Learn Express"
},
{
"id": 2,
"title": "Study Node"
}
]
PS E:\SEM 5\W&DF\Pra_4\task-manager-api> node server.js
Server running on port 5000
GET /tasks - 2026-07-31T09:21:12.027Z
POST /tasks - 2026-07-31T09:22:06.133Z
POST /tasks - 2026-07-31T09:22:31.751Z
GET /tasks - 2026-07-31T09:23:20.852Z

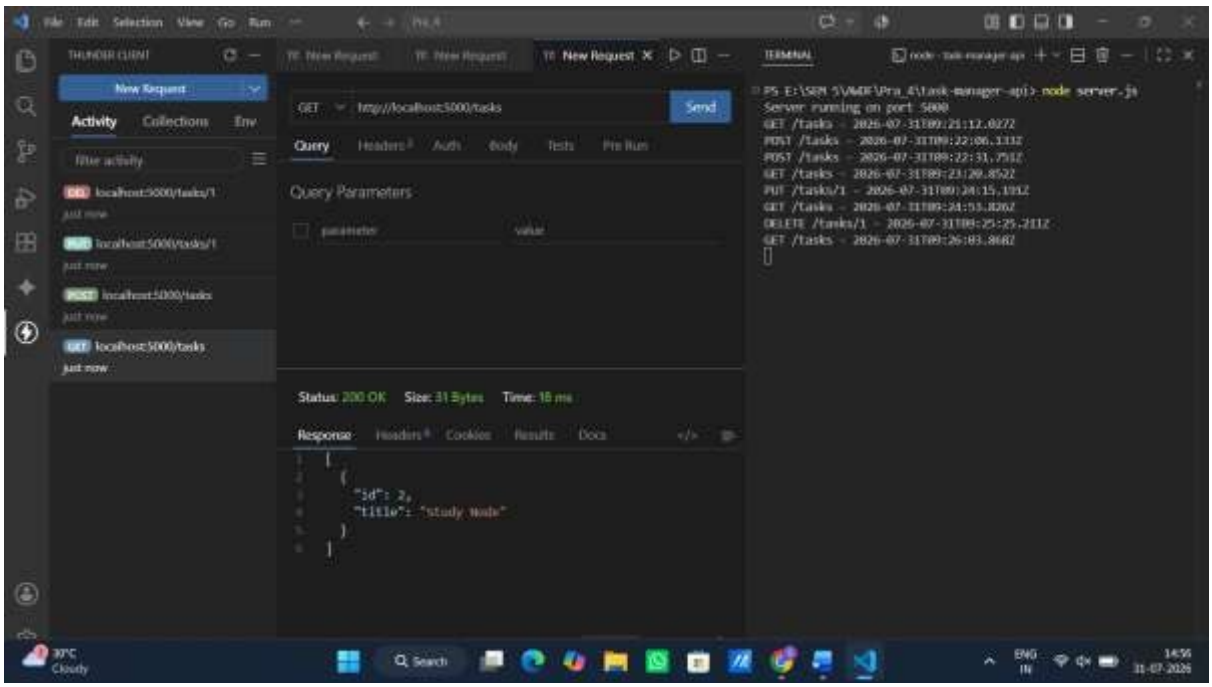
3) **PUT**


PUT http://localhost:5000/tasks/1
Send
{
"title": "Updated Task"
}
Status: 200 OK Size: 31 Bytes Time: 24 ms
{
"id": 1,
"title": "Updated Task"
}
PS E:\SEM 5\W&B\Pra_4\task-manager-api> node server.js
Server running on port 5000
GET /tasks - 2026-07-31T09:21:12.027Z
POST /tasks - 2026-07-31T09:22:06.133Z
POST /tasks - 2026-07-31T09:22:31.751Z
GET /tasks - 2026-07-31T09:23:20.852Z
PUT /tasks/1 - 2026-07-31T09:24:15.191Z

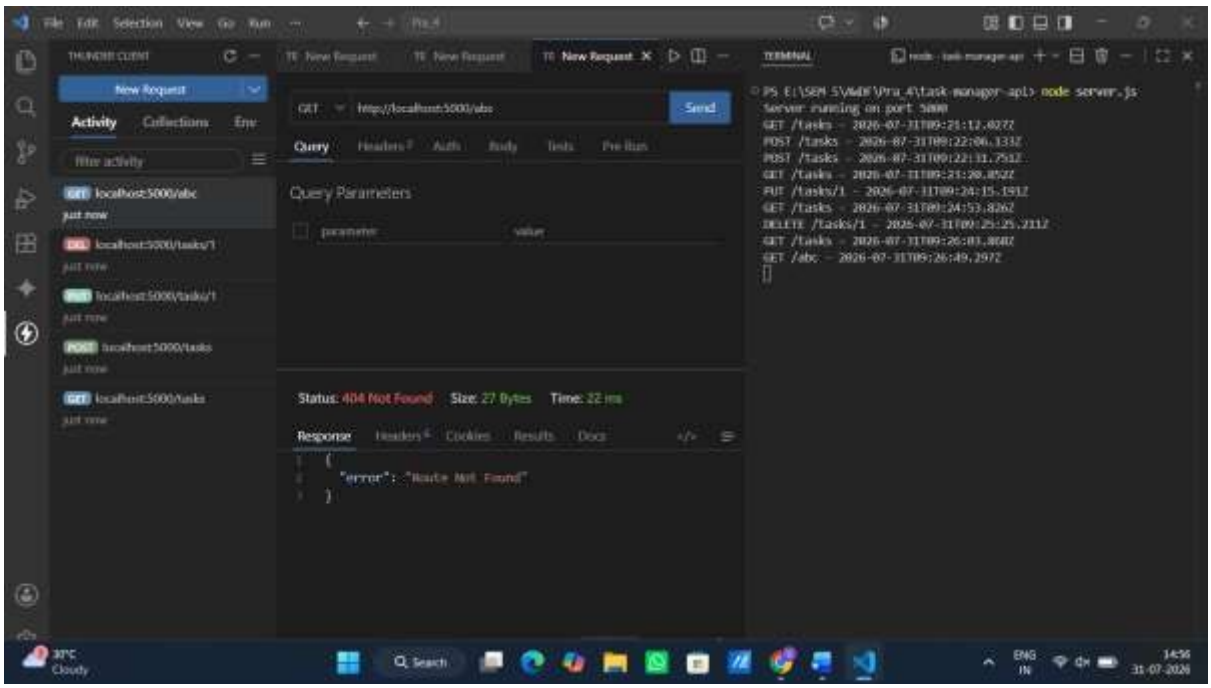

GET http://localhost:5000/tasks
Send
Query Parameters
Status: 200 OK Size: 63 Bytes Time: 29 ms
[
{
"id": 1,
"title": "Updated Task"
},
{
"id": 2,
"title": "study Node"
}
]
PS E:\SEM 5\W&B\Pra_4\task-manager-api> node server.js
Server running on port 5000
GET /tasks - 2026-07-31T09:21:12.027Z
POST /tasks - 2026-07-31T09:22:06.133Z
POST /tasks - 2026-07-31T09:22:31.751Z
GET /tasks - 2026-07-31T09:23:20.852Z
PUT /tasks/1 - 2026-07-31T09:24:15.191Z
GET /tasks - 2026-07-31T09:24:53.826Z

4) **DELETE**


DELETE
http://localhost:5000/tasks/1
Send
Query Parameters
Status: 200 OK Size: 39 Bytes Time: 20 ms
Response
{
"message": "task deleted successfully"
}
PS E:\SEM 5\W&DF\Pra_4\task-manager-api> node server.js
Server running on port 5000
GET /tasks - 2026-07-31T09:21:12.027Z
POST /tasks - 2026-07-31T09:22:06.133Z
POST /tasks - 2026-07-31T09:22:31.751Z
GET /tasks - 2026-07-31T09:23:20.852Z
PUT /tasks/1 - 2026-07-31T09:24:15.191Z
GET /tasks - 2026-07-31T09:24:53.826Z
DELETE /tasks/1 - 2026-07-31T09:25:25.211Z


GET
http://localhost:5000/tasks
Send
Query Parameters
Status: 200 OK Size: 31 Bytes Time: 18 ms
Response
[
{
"id": 2,
"title": "study Node"
}
]
PS E:\SEM 5\W&DF\Pra_4\task-manager-api> node server.js
Server running on port 5000
GET /tasks - 2026-07-31T09:21:12.027Z
POST /tasks - 2026-07-31T09:22:06.133Z
POST /tasks - 2026-07-31T09:22:31.751Z
GET /tasks - 2026-07-31T09:23:20.852Z
PUT /tasks/1 - 2026-07-31T09:24:15.191Z
GET /tasks - 2026-07-31T09:24:53.826Z
DELETE /tasks/1 - 2026-07-31T09:25:25.211Z
GET /tasks - 2026-07-31T09:26:03.868Z

5) **Verify 404 Handler**



Github Repo Link: https://github.com/sahilmak1602/WDF_24IT045_SAHIL.git

ASSIGNMENT – 1

To evaluate the Task Management REST API using the Richardson Maturity Model and determine its maturity level.

Theory: Richardson Maturity Model

The Richardson Maturity Model (RMM) is a framework used to evaluate how well a REST API follows REST principles. It consists of four maturity levels, from Level 0 to Level 3.

Level 0 – Single Endpoint (RPC Style)
At this level, the API uses a **single endpoint** for all operations. Different actions are identified through request parameters instead of separate URLs and HTTP methods.

Level 1 – Resources Using URIs
The API uses separate URLs (URIs) for different resources. For example, /tasks and /tasks/1 represent different resources, making the API more organized.

Level 2 – HTTP Methods and Status Codes
The API uses the correct HTTP methods such as GET, POST, PUT, and DELETE, along with appropriate HTTP status codes like 200 OK, 201 Created, 404 Not Found, and 500 Internal Server Error. Most modern REST APIs follow this level.

Level 3 – HATEOAS
The API includes HATEOAS (Hypermedia as the Engine of Application State) links in its responses. These links help clients discover available actions without hardcoding URLs.

Example:

```
{
  "id": 1,
  "title": "Task A",
  "_links": {
    "self": "/tasks/1",
    "delete": "/tasks/1"
  }
}
```

Richardson Maturity Model Evaluation

Level	Criterion	Satisfy?	Evidence
Level 0	Single endpoint	No	Multiple endpoints are used
Level 1	Resource URIs	Yes	/tasks, /tasks/:id
Level 2	HTTP methods & status codes	Yes	GET, POST, PUT, DELETE with 200, 201, 404, 500
Level 3	HATEOAS	No	Not implemented

API Endpoints

Method	Endpoint	Purpose
GET	/tasks	Get all tasks
POST	/tasks	Create task
PUT	/tasks/:id	Update task
DELETE	/tasks/:id	Delete task

Most production APIs stop at Level 2 because it provides a good balance between simplicity and REST principles. Clients usually know the API endpoints from documentation, so implementing HATEOAS adds complexity without significant benefit for many applications.

Conclusion:

The Task Management API successfully follows the principles of Richardson Maturity Model Level 2 by using resource-oriented URLs, appropriate HTTP methods (GET, POST, PUT, DELETE), and meaningful HTTP status codes. Since it does not provide HATEOAS links to guide clients, it does not qualify as a Level 3 REST API.